# Символы и строки

- символы char
- массив символов char[ ]
- неизменяемые строки string
- изменяемые строки StringBuilder
- регулярные выражения RegularExpressions

# Символы char

# Символьный тип *char*

- предназначен для хранения одного символа в двухбайтной кодировке *Unicode*
- Способы определения символьных констант:
  - символ, заключенный в одинарные кавычки
  - *escape*-последовательность, задающая код символа \x
  - *Unicode*-последовательность, задающая *Unicode*-код символа \u
- Пример 1

# Тип *char*

- относится к встроенным типам данных (*System.Char)*

| Метод | Описание |
| --- | --- |
| *GetNumericValue*() | Возвращает числовое значение символа, если он является цифрой, и –1 в противном случае |
| *GetUnicodeCategory*() | Возвращает категорию *Unicode*-символа (например, цифры, разделители строк, буквы в нижнем или верхнем регистре) |
| *Parse*() | Преобразует строку в символ (строка должна состоять из одного символа) |
| *ToLower*() | Преобразует символ в нижний регистр |
| *ToUpper*() | Преобразует символ в верхний регистр |

| Метод | Описание |
|---|---|
| *IsControl*() | Возвращает *true*, если символ является управляющим |
| *IsDigit*() | Возвращает *true*, если символ является десятичной цифрой |
| *IsLetter*() | Возвращает *true*, если символ является буквой |
| *IsLetterOrDigit*() | Возвращает *true*, если символ является буквой или десятичной цифрой |
| *IsLower*() | Возвращает *true*, если символ задан в нижнем регистре |
| *IsNumber*() | Возвращает *true*, если символ является цифрой (десятичной или шестнадцатеричной) |

| Метод | Описание |
| --- | --- |
| *IsPunctuation*() | Возвращает *true*, если символ является знаком препинания |
| *IsSeparator*() | Возвращает *true*, если символ является разделителем |
| *IsUpper*() | Возвращает *true*, если символ задан в нижнем регистре |
| *IsWhiteSpace*() | Возвращает *true*, если символ является пробельным (пробел, перевод строки, возврат каретки) |

- Примеры 2-4

# Массив символов *char*[]

- Базовый класс *Array* предоставляет для работы с массивом символов множество свойств и методов
- Используя динамический метод *ToCharArray*() класса *String*, можно выполнить преобразование из строки в массив символов (также можно посимвольно передать содержимое переменной *string* в массив символов)
- Примеры 5-6

# Неизменяемые строки string

- Каждый объект string – это неизменяемая последовательность символов Unicode, т.е. методы, предназначенные для изменения строк, возвращают измененные копии, исходные же строки остаются неизменными

# Способы создания

объявление ***с отложенной инициализацией***

```
string s1;
```

объявление ***с явной инициализацией***

*с неявным вызовом конструктора*

```
string s2 = "мёд ждём";
```

*с явным вызовом конструктора*

```
string s3 = new string('a', 10);
```

Пример 7

- Хотя строки относятся к ссылочным типам, при выполнении операций:
  - присваивание (=)
  - проверка эквивалентности (== или !=)
  - конкатенация (+)

работа происходит как со значимыми типами

- Строка - одномерный массив символов
  - обращение к любому символу строки происходит по индексу в квадратных скобках (нумерация с нуля)
  - обращаясь по индексу к символу строки можно лишь прочитать символ, но не изменить его
- Пример 8

- Типу *string*, который является встроенным типом *C#*, соответствует базовый класс *System.String* библиотеки *.NET*. Он предоставляет большой набор методов для работы со строками

# Свойства и методы класса

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **Compare()** | Статический метод | Сравнение двух строк в лексикографическом порядке (с учетом или без учета регистра) |
| **CompareTo()** | Метод | Сравнение текущего экземпляра строки с другой строкой |
| **Concat()** | Статический метод | Слияние произвольного числа строк |
| **Copy()** | Статический метод | Создание копии строки |
| **Format()** | Статический метод | Форматирование строки в соответствии с заданным форматом |

# Элементы класса

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **IndexOf(), IndexOfAny(), LastIndexOf(), LastIndexOfAny()** | Экземплярные методы | Определение индексов первого и последнего вхождения заданной подстроки или любого символа из заданного набора в данную строку |
| **Insert()** | Экземплярный метод | Вставка подстроки в заданную позицию |
| **Join()** | Статический метод | Слияние массива строк в единую строку, между элементами массива вставляются разделители |
| **Length** | Свойство | Длина строки |
| **PadLeft(), PadRight()** | Экземплярные методы | Выравнивают строки по левому или правому краю путем вставки нужного числа пробелов в начале или в конце строки |

# Методы работы со строками

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **Remove()** | Экземплярный метод | Удаление подстроки из заданной позиции |
| **Replace()** | Экземплярный метод | Замена всех вхождений заданной подстроки или символа новыми подстрокой или символом |
| **Split()** | Экземплярный метод | Разделяет строку на элементы, используя разные разделители. Результаты помещаются в массив строк |
| **StartsWith(), EndsWith()** | Экземплярные методы | Возвращают true или false в зависимости от того, начинается или заканчивается строка заданной подстрокой |
| **Substring()** | Экземплярный метод | Выделение подстроки, начиная с заданной позиции |

# Элементы класса

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **ToCharArray()** | Экземплярный метод | Преобразует строку в массив символов |
| **ToLower(), ToUpper()** | Экземплярные методы | Преобразование строки к нижнему или верхнему регистру |
| **Trim(), TrimStart(), TrimEnd()** | Экземплярные методы | Удаление пробелов в начале и конце строки или только с одного ее конца |

Вызов статических методов происходит через обращение к имени класса (String.Concat(str1, str2)),
в остальных случаях через обращение к экземплярам класса (str.ToLower())

- Каждый объект *string* – это неизменяемая последовательность символов, т. е. методы, предназначенные для изменения строк, возвращают в качестве результата новые строки, исходные же строки остаются неизменными.
- Примеры 9-11

# Изменяемые строки StringBuilder

- Класс *StringBuilder* определен в пространстве имен *System.Text*
- Объекты этого класса всегда инициализируются с явным вызовом конструктора класса (через операцию *new*)
- Конструктору можно передать две группы параметров
  - первая группа позволяет определить строку или подстроку, значением которой будет инициализироваться создаваемый объект
  - вторая группа параметров позволяет задать емкость объекта, т. е. объем памяти, отводимой данному экземпляру класса *StringBuilder*
  - Любая из этих групп не является обязательной и может быть опущена

# Примеры создания

- StringBuilder a =new StringBuilder();
- StringBuilder b = new StringBuilder("abcd");
- StringBuilder c = new StringBuilder(100);
- StringBuilder sb4 = new StringBuilder(10, 50);
- StringBuilder d = new StringBuilder("abcd", 100);
- StringBuilder d = new StringBuilder("abcd", 1, 2, 100);

# Объекты класса *StringBuilder* имеют две характеристики емкости

- текущую и максимальную

- В процессе работы текущая емкость может автоматически увеличиваться до максимальной емкости при увеличении размера строки
- Иногда бывает разумно уменьшать текущую емкость

- Над строками класса *StringBuilder* определены операции (работа со ссылками)
  - присваивания (=)
  - проверки эквивалентности (== и !=)
  - обращения к элементу по индексу ([])

но не определена операция конкатенации (+) (вместо нее существует метод *Append*(), дописывающий новую строку в конец существующей)

- Со строкой класса *StringBuilder* можно работать как с одномерным массивом символов, причем допускается не только чтение отдельного символа, но и его изменение
- Пример 1

# Свойства и методы класса StringBuilder

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **Append()** | Экземплярный метод | Добавление данных в конец строки |
| **AppendFormat()** | Экземплярный метод | Добавление форматированной строки в конец строки |
| **Capacity** | Изменяемое свойство | Емкость буфера (если устанавливаемое значение меньше текущей длины строки или больше максимального, то генерируется исключение *ArgumentOutOfRangeException*) |
| ***Chars*** | Изменяемое свойство | Возвращает из массива или устанавливает в массиве символ с заданным индексом. Вместо него можно пользоваться квадратными скобками [] |
| **CopyTo()** | Экземплярный метод | Копирует подмножество символов строки в массив символов |

# Свойства и методы класса StringBuilder

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **EnsureCapacity()** | Экземплярный метод | Изменение емкости буфера (если значение параметра меньше размера строки, то емкость устанавливается такой, чтобы гарантировать размещение строки) |
| **Equals()** | Экземплярный метод | Возвращает true, если объекты имеют одну и ту же длину и состоят из одних и тех же символов |
| **Insert()** | Экземплярный метод | Вставка подстроки с заданной позиции |
| **Length** | Изменяемое свойство | Длина строки. Присвоение ему значения 0 сбрасывает содержимое и очищает строку |
| **MaxCapacity** | Неизменяемое свойство | Возвращает наибольшее количество символов, которое может быть размещено в строке |

# Свойства и методы класса StringBuilder

| *Название* | *Вид* | *Описание* |
|---|---|---|
| **Remove()** | Экземплярный метод | Удаление подстроки с заданной позиции |
| **Replace()** | Экземплярный метод | Замена всех вхождений заданной подстроки или символа новой подстрокой или символом |
| **ToString()** | Экземплярный метод | Преобразование в строку типа *string* |

- Пример 2

- На практике часто комбинируют использование изменяемых и неизменяемых строк, при этом работа со строками происходит следующим образом: сначала конструируется строка класса *StringBuilder*, над ней выполняются операции, требующие изменение значения; затем полученная строка преобразуется в строку класса *String* и над ней выполняются операции, не требующие изменения значения строки
- Пример 4

# Регулярные выражения *RegularExpression*

using System.Text.RegularExpressions;

# Пространство имен *RegularExpression*

- предоставляет специальный мощный аппарат работы со строками в *.NET Framework*
- содержит набор классов, обеспечивающих работу с регулярными выражениями
- в основе работы лежит использование строки-образца (*pattern*, шаблона, маски), задающей правила поиска
- В пространстве *RegularExpression* расположено одно перечисление (*RegexOptions*) и восемь связанных между собой классов (*Regex*, *Match*, *MatchCollection*, *Group*, *GroupCollection*, *Capture*, *CaptureCollection* и *RegexCompilationInfo*)
- Пример 5